



Grado en Ingeniería Informática
Arquitectura de Computadores 25/26
Grupo 81

Proyecto de programación orientada al rendimiento
«Aplicación de síntesis de imágenes 3D»

Héctor Molina Garde — 100522253
Noa López Fernández — 100522230
Guillermo González Avilés — 100522146
Nicolás Maire Bravo — 100522100

Equipo 10

Profesor
José Daniel García Sánchez

Tabla de Contenidos

1. Diseño	2
1.1. Diseño de la estructura	2
1.2. Diseño conceptual	3
1.3. Diseño de comportamiento	3
2. Optimización	5
2.1. Precálculo de operaciones en los cilindros.	5
2.2. Axis-Aligned Bounding Box. (AABB)	5
2.3. Modificación de flags.	5
3. Pruebas realizadas	6
3.1. Pruebas unitarias	6
3.2. Pruebas funcionales de sistema	6
4. Evaluación rendimiento y energía	7
4.1. Evaluación del rendimiento	7
4.1.1. Comparativa de rendimiento en base a las figuras y sus materiales.	8
4.1.2. Comparativa de rendimiento en base a las cualidades de la imagen generada.	9
4.2. Evaluación de la energía	10
4.2.1. Comparativa de rendimiento en base a las figuras y sus materiales.	10
4.2.2. Comparativa de rendimiento en base a las cualidades de la imagen generada.	10
4.3. Conclusiones finales	11
5. Organización de trabajo	12
5.1. División de trabajo	12
5.2. Desarrollo de tareas	13
6. Conclusiones	14
7. Uso de herramientas de IA	16
8. Bibliografía	17

1. Diseño

Esta sección detalla el diseño de la aplicación, creado para soportar **las estrategias de datos SOA (*Structure of Arrays*) y AOS (*Array of Structures*)**, analizando su arquitectura, modelo conceptual y flujo de ejecución.

1.1. Diseño de la estructura

El diseño estructural define los módulos principales del software, incluyendo sus responsabilidades y relaciones con otros módulos del proyecto.

La arquitectura sigue un principio de separación de responsabilidades. De tal manera que se aíslan las implementaciones específicas de SOA y AOS de la lógica común que es utilizada por ambas.

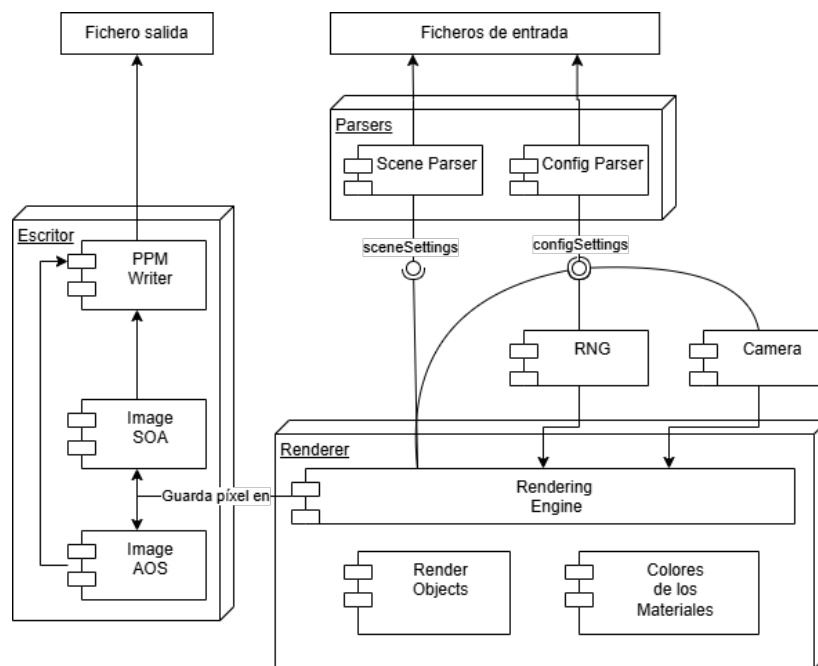


Figura 1: Diagrama de componentes de la arquitectura.

Los componentes principales identificados en la figura **Figura 1** son:

- **Módulo common:** Agrupa toda la lógica y las estructuras de datos que no dependen de la estrategia de almacenamiento de la imagen. Incluye:
 - Definiciones matemáticas (vectores, rayos, colores).
 - Lógica de intersección de rayos con objetos (esferas, cilindros).
 - Gestión de materiales y cálculo de reflexiones/refracciones.
 - El parser para los archivos de configuración y de escena.
- **Módulo aos:** Contiene la implementación específica para la estrategia **Array of Structures**. La principal diferencia reside en la estructura de datos que representa la imagen en memoria y el ejecutable `render-aos`.
- **Módulo soa:** Análogamente, contiene la implementación para la estrategia **Structure of Arrays**. Su estructura de imagen se compone de tres vectores separados para los canales R, G y B. También provee el ejecutable `render-soa`.

- **Ejecutables (`render-aos` y `render-soa`):** Son los puntos de entrada de la aplicación. Su responsabilidad es gestionar los argumentos de la línea de comandos, orquestar el proceso de parseo y renderizado, y finalmente generar el archivo de salida en formato PPM.

1.2. Diseño conceptual

El diseño conceptual se centra en las abstracciones y entidades clave que modelan el problema del renderizado por trazado de rayos. Estas clases y estructuras de datos forman el núcleo de la lógica de la aplicación.

Placeholder: Diagrama de Clases del Dominio

Figura 2: Modelo conceptual de las entidades principales del sistema.

A continuación, se describen las responsabilidades de las abstracciones más relevantes mostradas en la figura [Figura 2](#):

- **Objeto:** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat.

1.3. Diseño de comportamiento

El diseño de comportamiento describe cómo los objetos y componentes interactúan a lo largo del tiempo para llevar a cabo las tareas principales. El flujo principal de la aplicación se puede dividir en dos fases: la carga de la configuración y el bucle de *renderizado*.

Placeholder: Diagrama de Secuencia del Bucle de Renderizado

Figura 3: Diagrama de secuencia que ilustra el proceso de renderizado para un único píxel.

El proceso completo, ilustrado en la figura [Figura 3](#), sigue los siguientes pasos:

1. **Inicialización:** El programa principal (`main`) recibe las rutas de los archivos de configuración, escena y salida. Se instancia un `Parser` para leer y validar estos archivos, creando los objetos `Scene` y `Camera`.

2. **Bucle principal de renderizado:** El programa itera sobre cada píxel de la imagen de salida (de arriba a abajo, de izquierda a derecha).
3. **Muestreo por píxel (Anti-aliasing):** Para cada píxel, se inicia un bucle de muestreo. Se lanzan múltiples rayos (`samples_per_pixel`) a través de posiciones aleatorias dentro del área del píxel para suavizar los bordes.
4. **Trazado de rayo:** Para cada rayo muestreado, se invoca a una función recursiva `ray_color()`. Esta función:
 - Calcula la intersección más cercana del rayo con los objetos de la `Scene`.
 - Si no hay intersección, devuelve el color de fondo.
 - Si hay intersección, consulta el `Material` del objeto golpeado para determinar cómo el rayo es atenuado y si es reflejado o refractado.
 - Se genera un nuevo rayo y se llama recursivamente a `ray_color()` con una profundidad reducida. El proceso se detiene al alcanzar la profundidad máxima (`max_depth`).
5. **Acumulación y promedio:** El color resultante de cada rayo muestreado se acumula. Al final del bucle de muestreo, el color total se divide por el número de muestras para obtener el color final del píxel.
6. **Corrección y almacenamiento:** Se aplica la corrección gamma al color promediado. El valor de color final (en formato de punto flotante `[0,1]`) se escala al rango de enteros `[0, 255]` y se almacena en la estructura de datos de la `Image` (ya sea en formato AOS o SOA).
7. **Generación del archivo de salida:** Una vez procesados todos los píxeles, el contenido de la estructura `Image` se escribe en el archivo de salida siguiendo el formato PPM (P3).

2. Optimización

2.1. Precálculo de operaciones en los cilindros.

Tras las primeras pruebas de rendimiento de nuestro programa, pudimos observar que un gran porcentaje del tiempo dedicado a la ejecución estaba concentrada en torno a la renderización de los cilindros (abarcando $\sim 67\%$ de las instrucciones ejecutadas), por tanto, la gran mayoría de nuestros primeros esfuerzos fue a optimizar el renderizado de cilindros.

Así, la primera solución encontrada fue precalcular las operaciones con un coste computacional elevado que se realizaban durante las intersecciones y generaciones de rayos con las diferentes partes de los cilindros (raíces cuadradas etc.). De este modo, conseguimos una reducción significativa en el número de veces que se realizaban dichas operaciones, y por tanto una reducción significativa en el tiempo de ejecución del programa. No obstante, esto también nos presentó inconvenientes y nuevos conceptos en los que profundizar, como es la gestión de la memoria caché. Pese a no presentar un problema muy amplio en nuestro código, fue una cuestión a tener en cuenta de cara a las siguientes optimizaciones, pues dado que ya habíamos realizado un precálculo de las operaciones más costosas en los cilindros, nos abstenimos de realizarlas en otras secciones de nuestro proyecto dado que podría generar fallos de memoria caché y por tanto, una pérdida de rendimiento mayor a la que genera el cálculo que pretendíamos optimizar.

2.2. Axis-Aligned Bounding Box. (AABB)

También se ha planteado un modelo AABB (Axis-Aligned Bounding Box), cuyo uso es muy frecuente en el renderizado de videojuegos. El método consiste en introducir los elementos gráficos en una «caja» tridimensional alineada con los ejes de coordenadas. Dicha caja se utiliza para realizar una primera aproximación de las colisiones entre rayos, en caso de que el rayo se introduzca en el espacio delimitado por la caja, se comprueba si realmente el rayo colisiona con el objeto, en caso contrario, no se calcula la intersección.

Esto resulta eficiente ya que el cálculo de intersecciones de los rayos con los cilindros, y especialmente con las esferas, requieren numerosas operaciones con un coste computacional bastante elevado como ya se ha mencionado en el apartado anterior. Sin embargo, las intersecciones con la caja que envuelve los objetos, dadas sus características y su condición alineada con los ejes de coordenadas, requieren operaciones con un coste computacional muy reducido (mayoritariamente sumas). Por tanto, pese a requerir una comprobación previa que puede generar una pequeña pérdida de rendimiento en los casos en los que los rayos sí impacten con los objetos, en la gran mayoría de los casos, se evita realizar las operaciones con un coste más elevado ya que se descarta la posibilidad de intersección con las operaciones más sencillas de la caja AABB que rodea los elementos, haciendo que el rendimiento general del programa mejore notoriamente. Además es un modelo que presenta una muy buena escalabilidad, pues sus efectos, como ya se demostrará en apartados posteriores del documento, tiene un mayor efecto en el rendimiento cuantos más elementos gráficos haya en la escena.

2.3. Modificación de flags.

3. Pruebas realizadas

Para comprobar que el correcto funcionamiento del código presentado hemos realizado pruebas de dos tipos principales que se presentan a continuación.

3.1. Pruebas unitarias

En estas pruebas se realiza una comprobación del funcionamiento unitario de las funciones que conforman las diferentes clases del repositorio. Para ello, hemos empleado las herramientas de *Google Test* como indicado en clase.

3.2. Pruebas funcionales de sistema

En las pruebas funcionales del sistema se comprueba, no el funcionamiento de funciones de manera individual y aislada, si no el funcionamiento global del programa, comprobando que realiza las operaciones como se espera y que se obtienen los resultado generales esperados.

4. Evaluación rendimiento y energía

En esta sección realizaremos un análisis del rendimiento y de la energía. EN nuestro caso, la analizaremos en base a diferentes factores, entre ellos: el tipo de figura y su material y las características de la imagen: tamaño, profundidad y numero de muestras por pixel. De este modo, hemos realizado comprobaciones de los tiempos de ejecución y energía consumida para diferentes valores de las métricas mencionadas. Los datos resultantes, además de poder encontrarse en el repositorio en una carpeta que contiene los srulms y escenas analizadas, podemos encontrarlos en la hoja de cálculo que podemos encontrar en el siguiente link: [Evaluación de rendimiento y energía..](#)

Un apunte previo a realizar es el hecho de que para las gráficas de estudio de las esferas y cilindros, tanto en el caso de estudio de rendimiento como en el de energía, dado que el número de elementos a analizar se ha planteado de manera exponencial (estudiamos el incremento con 1, 2, 4 etc. figuras) los datos se presentan con un ajuste logarítmico en el eje de ordenadas la métrica no es el tiempo si no el logaritmo en base dos del tiempo o energía consumida. De esta manera, podemos entender debidamente el patrón de crecimiento de nuestro problema, pues ambos ejes están ajustados a sus respectivos valores. No obstante, los valores que se mencionarán en el análisis será el valor real, no el del ajuste logarítmico.

4.1. Evaluación del rendimiento

Para comenzar con el estudio, presentamos la siguiente gráfica que analiza la diferencia de tiempos de ejecución entre nuestras escenas y las escenas de referencia presentadas.

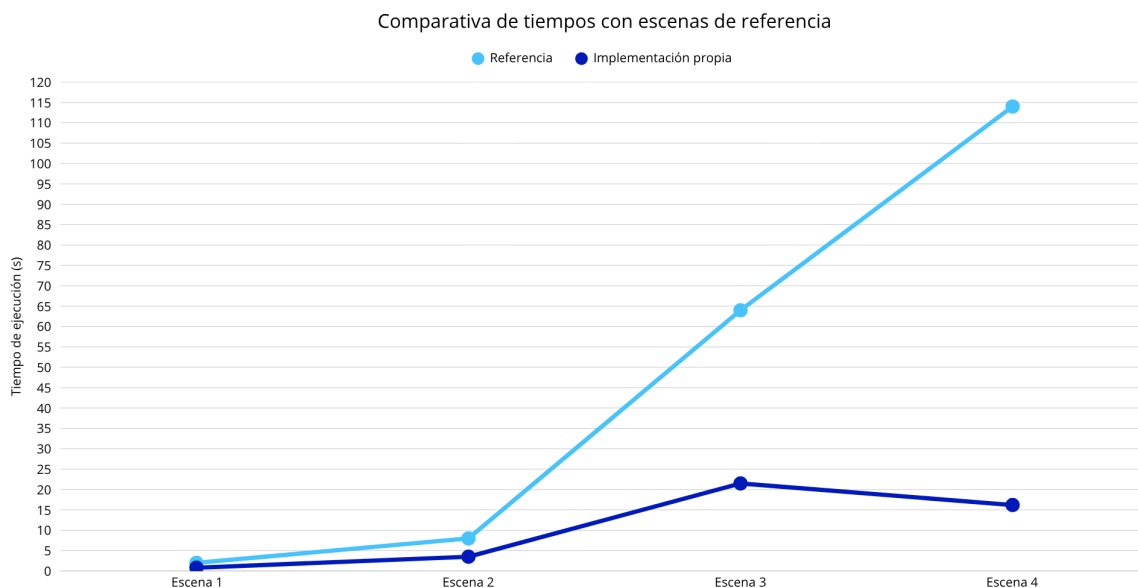


Figura 4: Gráfica comparativa de imágenes de referencia e implementación propia.

Como se observa en la gráfica, los patrones de comportamiento entre nuestra implementación y los valores de referencia son completamente diferentes. Esta divergencia no solo se manifiesta en la disparidad de valores, en los que podemos observar una clara reducción con respecto a los de referencia en los nuestros, sino también en la velocidad de crecimiento respecto a las escenas, que es significativamente más lenta en nuestro caso. De hecho, mientras la imagen de referencia muestra un crecimiento continuo en el tiempo de ejecución, nuestra implementación presenta una reducción en la cuarta escena. Esto se debe a que nuestro modelo de optimización AABB, pese a producir una mejora del rendimiento en todos los casos, es más destacable a mayor sea el número de elementos de la imagen, lo que presenta una buena escalabilidad de nuestro modelo.

4.1.1. Comparativa de rendimiento en base a las figuras y sus materiales.

Para esta comparación analizando las diferencias entre materiales y técnicas de gestión de arrays hemos planteado diferentes evaluaciones en las que presentamos

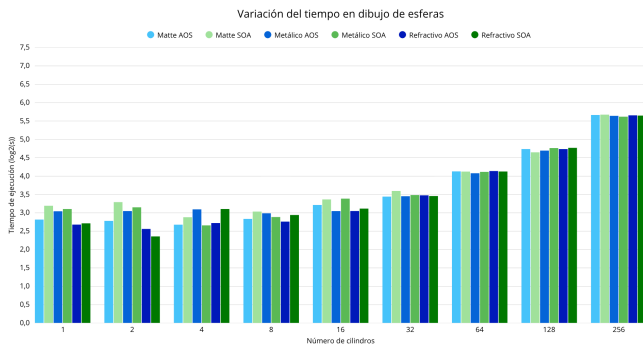


Figura 6: Tiempo de ejecución en esferas

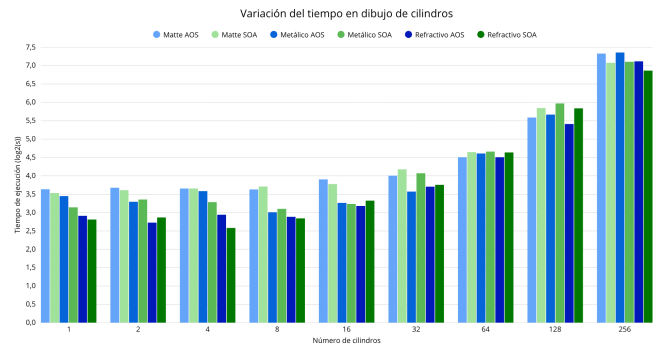


Figura 7: Tiempo de ejecución en cilindros

De estas gráficas se pueden sacar diferentes análisis y conclusiones. Para su mejor comprensión, comenzaremos por las conclusiones más evidentes y generales para luego ir profundizando en los detalles.

La primera cosa a mencionar es el patrón de crecimiento del tiempo de ejecución en función de números de elementos de la imagen. Tanto en el caso de las esferas como en el de los cilindros, se observa un crecimiento relativamente lineal. Esto, demuestra que nuestro código presenta una buena escalabilidad, ya que el tiempo de ejecución crece de manera controlada y predecible.

Además, los cilindros ofrecen, de manera general, tiempos de ejecución superiores a los de las esferas. Presentando entre 2 segundos y 25 segundos de diferencia. Esto, tiene perfecto sentido, ya que las operaciones requeridas para el cálculo de los cilindros y sus intersecciones son de mayor complejidad. De hecho, en los análisis intermedios realizados durante el proceso de la implementación, ya se observaba esta diferencia de rendimiento entre ambas figuras, con un punto del proyecto en el que el 67% del tiempo de ejecución se dedicaba a la generación de cilindros.

Además podemos observar en ambos casos diferencias en el tiempo de ejecución en función del empleo de un planteamiento AOS o SOA. Basándose en las gráficas, es complejo determinar exactamente los efectos de dichos modelos en los tiempos de ejecución, pues los patrones no son consistentes. No obstante, podríamos decir que los tiempos de ejecución usando *SOA* son ligeramente superiores a los del modelo *AOS* aunque las diferencias mínimas y, en términos generales, el patrón de crecimiento es similar en ambos casos.

Finalmente, en cuanto a los materiales podemos observar diferentes tendencias. En el caso de las esferas, el tiempo de ejecución entre materiales es mucho más similar, presentando menor variación. Es posible que el material que menor tiempo de ejecución requiere sea el material refractivo, cosa bastante sorprendente, dado que los cálculos y el entendimiento de la refracción nos ha resultado más compleja de comprender. En el caso de los cilindros, resulta muy visual ver como los materiales metálicos resultan en un mayor tiempo de ejecución, los refractivos el menor y los metálicos el mayor tiempo de ejecución.

De este modo, podemos determinar que en general, las tendencias de tiempo de ejecución son realmente similares entre cilindros y esferas, aunque los tiempos resulten ligeramente superiores para los cilindros. De la misma forma, tampoco se aprecia una diferencia realmente significativa entre el modelo *AOS* y el modelo *SOA*. En cualquier caso, pese a haber ligeras diferencias, el material no parece tener un impacto significativo en el tiempo de ejecución, resultando bastante similar en todos los materiales y siendo variado en todos los casos.

4.1.2. Comparativa de rendimiento en base a las cualidades de la imagen generada.

Para realizar una estimación del efecto de las cualidades de la imagen en el tiempo de ejecución hemos realizado tres gráficas diferentes que analizan el efecto del tamaño, la calidad de la imagen (muestras por pixel) y la profundidad máxima de recursión.

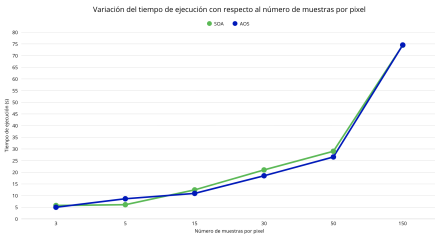


Figura 9: Tiempo de ejecución en función de las muestras por pixel

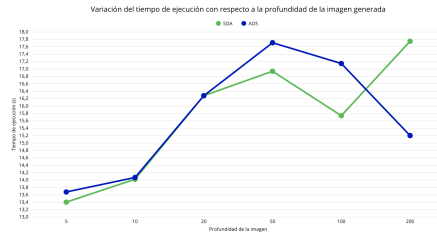


Figura 10: Tiempo de ejecución en función de la profundidad

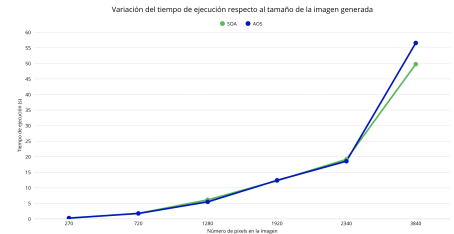


Figura 11: Tiempo de ejecución en función del tamaño

- **Muestras por pixel:** Como se podría esperar, el número de muestras por pixel, que define la calidad de la imagen, tiene un impacto directo en el tiempo de ejecución. En la gráfica se puede observar como el tiempo de ejecución crece de manera aproximadamente exponencial conforme aumentamos el número de muestras por pixel de una manera practicamente idéntica en la versión de AOS y de SOA. Esto tiene sentido, ya que a mayor número de muestras por pixel, mayor es el número de rayos que deben intervenir, lo que incrementa el cálculo necesario de las intersecciones y, por tanto, el tiempo de ejecución total.
- **Tamaño de la imagen:** El tamaño de la imagen también tiene un impacto similar de caracter exponencial en el tiempo de ejecución, también practicamente idéntico entre AOS y SOA. Este comportamiento también es el esperado, dado que a más píxeles, más cosas se deben procesar y calcular en la imagen final, por tanto es simplemente lógico que el tiempo de ejecución crezca de manera exponencial conforme aumentamos el tamaño de la imagen.
- **Profundidad:** La profundidad, por otro lado, presenta un comportamiento completamente diferente, ya no solo entre AOS y SOA sino también en el patrón de crecimiento. Como se puede ver en la gráfica, por un lado, en los casos representados, el AOS presenta un crecimiento con una pendiente muy grande hasta alcanzar un punto máximo al alcanzar la profundidad 50 en la imagen, reduciendo el tiempo de ejecución de manera persistente a partir de ese punto. Por otro lado, en el SOA también se produce un crecimiento similar hasta el máximo también en la profundidad 50, para reducirse en la profundidad con valor 100 y crecer de nuevo de ahí en adelante. Este comportamiento es bastante extraño y no resulta nada sencillo de explicar. Nuestra hipótesis podría ser que una vez aumentamos la profundidad llega un punto en el que los rayos de las propias imágenes ya no contribuyen de manera tan significativa en el resultado final de la escena. Por ello, nos resulta complejo explicar el crecimiento final en el caso del modelo SOA.

En resumen y de manera completamente general, podemos concluir que el tiempo de ejecución crece influenciado mayoritariamente por los factores de muestras por pixel y tamaño de la imagen, dado que estos crecen de manera más concreta y predecible presentando un aumento exponencial en el tiempo de ejecución. El efecto de la profundidad en lo mismo resulta más complejo de entender y presenta un patrón de crecimiento menos predecible. Por ello, para sacar unas conclusiones más concreto sería necesario realizar pruebas más complejas, combinando el factor junto con otros factores para entender mejor su impacto en casos menos aislados y concretos. No obstante, como en los casos anteriores, nuestros modelos, probablemente porque el modelo de optimización utilizado presenta una solución eficiente, tanto el AOS como el SOA no presentan diferencias significativas en los tiempos de ejecución, presentando en todos los casos de estudio comportamientos muy similares que no nos ayudan a decantarnos por un modelo u otro.

4.2. Evaluación de la energía

4.2.1. Comparativa de rendimiento en base a las figuras y sus materiales.

Las gráficas resultantes en este caso, del mismo modo que en el anterior y presentando una escala logarítmica, son las que se pueden observar a continuación:

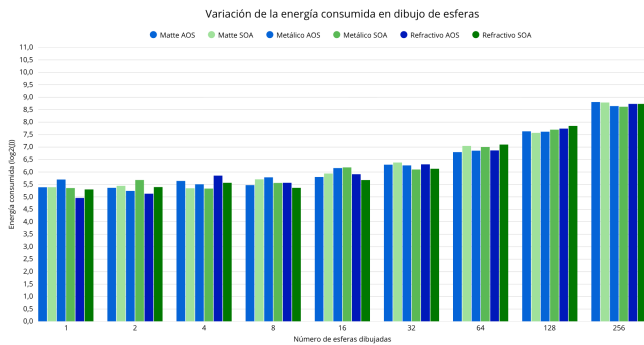


Figura 13: Consumo de energía en esferas

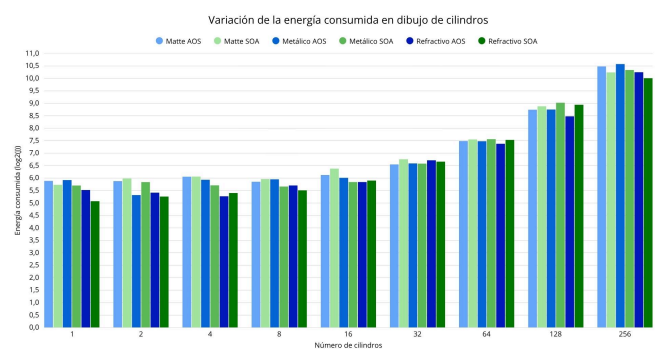


Figura 14: Consumo de energía en cilindros

Los comentarios a hacer son realmente similares, pues como se observa la forma y crecimiento de estas gráficas es prácticamente idéntica a las del tiempo de ejecución. Esto, deja entrever que ambas están fuertemente relacionadas, es decir, a mayor tiempo de ejecución también es mayor el consumo energético. Además, un apunte a realizar que no se ha realizado con anterioridad y que también se puede ver en el caso del tiempo de ejecución, es el hecho de que en ambos casos, la variación de energía para los primeros casos (de 2 objetos a 16) el incremento es prácticamente nulo y todos presentan un consumo energético muy similar. Esto, puede deberse a que en estos casos el consumo energético base del programa es mucho mayor que el incremento producido por la adición de nuevos objetos, por lo que no se aprecia un incremento real en el consumo energético hasta que el número de objetos es suficientemente grande.

Por lo demás, como ya hemos mencionado en el caso anterior, el consumo energético de los cilindros es mayor que el de las esferas, presentando un patrón de crecimiento similar en ambos casos. Además, tampoco se aprecian diferencias significativas entre el modelo AOS y SOA, presentando ambos patrones de crecimiento similares y sin diferencias notables en los valores.

4.2.2. Comparativa de rendimiento en base a las cualidades de la imagen generada.

Para finalizar el estudio de la energía consumida, presentamos las gráficas resultantes del análisis de las cualidades de la imagen en el consumo energético.

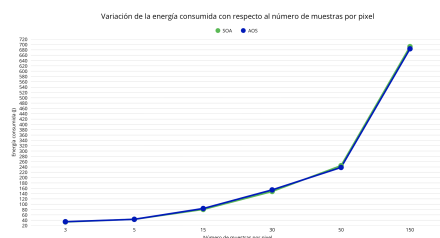


Figura 16: Consumo de energía en función de las muestras por pixel

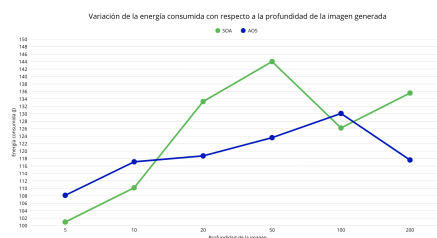


Figura 17: Consumo de energía en función de la profundidad

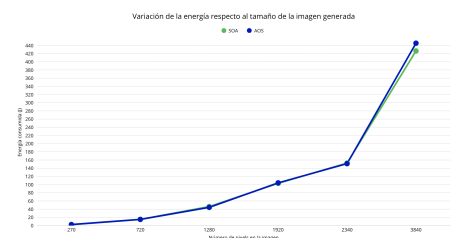


Figura 18: Consumo de energía en función del tamaño

En las gráficas presentadas se puede observar, como en el caso del tiempo de ejecución, un patrón de crecimiento prácticamente idéntico entre los modelos AOS y SOA, que se prepresenta de manera muy similar a un incremento exponencial, recalcando la ya mencionada relación entre el tiempo de ejecución y el consumo energético.

Sin embargo, como en el caso anterior el modelo de profundidad presenta un comportamiento diferente. De hecho, el comportamiento que presenta es también diferente al del tiempo de ejecución. En el caso del SOA la similaridad es mayor, no obstante, en el caso del AOS el patrón de crecimiento es realmente diferente, presentando un incremento de la energía consumida mucho más lento y controlado que en el caso del tiempo de ejecución que se mantiene en el tiempo cayendo ligeramente incluso en los casos de profundidad más elevada. De este modo, en este caso si podemos concluir prácticamente con total seguridad que para casos de profundidades elevadas el modelo *AOS* presenta una mejor eficiencia energética, y más importante, mayor escalabilidad dado su incremento más reducido.

4.3. Conclusiones finales

En definitiva, dados los datos presentados y las conclusiones extraídas, podemos concluir que nuestro modelo de optimización AABB (Axis Aligned Bounding Box) presenta una buena escalabilidad y eficiencia tanto en tiempo de ejecución como en consumo energético. Esto lo sabemos principalmente por los datos lineales de incremento en función del número de elementos. En definitiva, este método de optimización pretende optimizar los casos en los que hay mucho flujo de figuras, reduciendo el cálculo de intersecciones innecesarias. Además, podemos concluir que los factores que más afectan al tiempo de ejecución y consumo energético en relación a la configuración de la imagen son el tamaño de la imagen y el número de muestras por pixel, ambos con un crecimiento exponencial en ambas métricas. Por otro lado, la profundidad presenta un comportamiento más complejo y menos predecible.

Finalmente, en cuanto a la comparación entre los modelos *AOS* y *SOA*, podemos concluir que ambos presentan comportamientos muy similares en la mayoría de los casos, sin diferencias significativas en tiempo de ejecución o consumo energético. Esto seguramente tenga relación con la aplicación de nuestros métodos de optimización. No obstante, en el caso del consumo energético en función de la profundidad, el modelo *AOS* presenta un comportamiento más eficiente y escalable, lo que podría hacerlo más adecuado para escenas con altas profundidades.

5. Organización de trabajo

5.1. División de trabajo

Para garantizar la entrega a tiempo del proyecto en cuestión, primero se ha desarrollado una estructura lógica del tiempo disponible y dividido el desarrollo del proyecto en 4 secciones de trabajo que se han organizado de manera que bien podría recordar a un proyecto de Desarrollo Agile, en el que se subdividen etapas y se realizan entregas continuas de pequeñas tareas hasta completar dichas secciones. Para ello, hemos empleado la herramienta **Jira**, que ha permitido la organización en **Sprints** (pequeñas tareas) de las diferentes secciones de trabajo.

La subdivisión en secciones de trabajo y sus correspondientes sprints se incluyen a continuación:

1. **Diseño:** Durante la etapa, se realizó una investigación activa y concreta de recursos, modelos y conceptos que se usan en el ámbito del renderizado de imágenes y posibles formas de optimización del mismo que pudieran resultar eficientes para la tarea concreta. También se desarrollo el diseño general del código a realizar. Notese como en este sección no se programó en absoluto, simplemente organizamos los pasos a seguir para una programación efectiva y rápida.

☐	Actividad	Persona asignada	Informador	Fecha de vencimiento	Creada	Prioridad
☐	☞ SECCIÓN DE DISEÑO	Sin asignar	NF Noa López Ferná...	06 oct 2025	30 sept 2025, 11:51	Medium
☐	☑ Entender correctamente los modelos matemáticos	Sin asignar	NF Noa López Ferná...	06 oct 2025	30 sept 2025, 11:55	Medium
☐	☑ Búsqueda de métodos de optimización	Sin asignar	NF Noa López Ferná...	06 oct 2025	30 sept 2025, 11:54	Medium
☐	☑ Preparar entorno para la práctica	100522146	NF Noa López Ferná...	06 oct 2025	30 sept 2025, 11:53	Medium
☐	☑ Modelización de diseño	NF Noa López Fernández	NF Noa López Ferná...	06 oct 2025	30 sept 2025, 11:52	Medium
☐	☑ Búsqueda de información sobre programación o...	100522100	NF Noa López Ferná...	06 oct 2025	30 sept 2025, 11:51	Medium

Figura 19: Sprint 1 de tareas: Diseño.

2. **Desarrollo funcional:** En esta etapa, la misión principal fue asegurar que conseguíamos representar esferas y cilindros, sus materiales y conseguíamos generar imágenes .ppm de manera correcta. El objetivo principal de la sección no era tanto conseguir elementos funcionales, optimizados y eficientes, si no garantizar la validez del diseño realizado y la generación de un producto mínimo para poder aplicar las optimizaciones, o lo que es lo mismo, garantizar que al menos se pueden generar las imágenes y elementos correctamente.

☐	Actividad	Persona asignada	Informador	Fecha de vencimiento	Creada	Prioridad
☐	☞ SECCIÓN DE PROGRAMACIÓN	Sin asignar	NF Noa López Ferná...	13 oct 2025	07 oct 2025, 7:29	Medium
☐	☑ Rayos BVH	HM Héctor Molina	NF Noa López Ferná...	13 oct 2025	07 oct 2025, 7:34	Medium
☐	☑ Creación escena	HM Héctor Molina	NF Noa López Ferná...	13 oct 2025	07 oct 2025, 7:34	Medium
☐	☑ Cálculo de Colores	NF Noa López Fernández	NF Noa López Ferná...	13 oct 2025	07 oct 2025, 7:34	Medium
☐	☑ Materiales (Mate, Reflectivo, Metálico)	100522146	NF Noa López Ferná...	13 oct 2025	07 oct 2025, 7:33	Medium
☐	☑ Post Procesado	NF Noa López Fernández	NF Noa López Ferná...	13 oct 2025	07 oct 2025, 7:32	Medium
☐	☑ Intersecciones de rayos	NF Noa López Fernández	NF Noa López Ferná...	13 oct 2025	07 oct 2025, 7:32	Medium
☐	☑ Trazado de rayos	HM Héctor Molina	NF Noa López Ferná...	13 oct 2025	07 oct 2025, 7:31	Medium
☐	☑ Escritura de imagen SOA Y AOS	NF Noa López Fernández	NF Noa López Ferná...	13 oct 2025	07 oct 2025, 7:31	Medium
☐	☑ Parsers, errores etc.	100522146	NF Noa López Ferná...	13 oct 2025	07 oct 2025, 7:31	Medium

Figura 20: Sprint 2 de tareas: Desarrollo.

3. **Optimización y pruebas:** Esta sección resulta de las más relevantes del proyecto. En ella, se trabaja sobre lo obtenido en la sección anterior para optimizar los resultados, garantizando la eficiencia energética y la optimización del tiempo de ejecución. Para ello, se utilizan las técnicas descubiertas durante la fase de diseño y se aplican al código desarrollado durante la fase de desarrollo. En esta fase también se realizan las pruebas del programa, tanto unitarias como de rendimiento y energía. En definitiva, se garantiza que el código desarrollado es eficiente, correcto y cumple con los estándares mínimos de calidad y eficiencia requeridos.

☐	Actividad	Persona asignada	Informador	Fecha de vencimie...	Creada	↓	Prioridad
☐	✦ KAN-17 OPTIMIZACIONES Y PRUEBAS	Sin asignar	Noa López Fern...	24 oct 2025	20 oct 2025, 6:21		⚖ Medium
☐	☒ KAN-24 Corrección de errores	Sin asignar	Noa López Fern...	24 oct 2025	20 oct 2025, 6:27		⚖ Medium
☐	☒ KAN-23 Solución de errores de clang tidy y formato	Sin asignar	Noa López Fern...	24 oct 2025	20 oct 2025, 6:25		⚖ Medium
☐	☒ KAN-22 Pruebas de rendimiento y energía	Sin asignar	Noa López Fern...	24 oct 2025	20 oct 2025, 6:25		⚖ Medium
☐	☒ KAN-21 Pruebas unitarias	Sin asignar	Noa López Fern...	24 oct 2025	20 oct 2025, 6:23		⚖ Medium
☐	☒ KAN-20 Optimizar cilindros (gastan mucho) ↗ +	100522146	Noa López Fern...	24 oct 2025	20 oct 2025, 6:23		⚖ Medium
☐	☒ KAN-19 Implementar AABB	Noa López Fernández	Noa López Fern...	24 oct 2025	20 oct 2025, 6:23		⚖ Medium
☐	☒ KAN-18 Realizar estudio del rendimiento	Héctor Molina	Noa López Fern...	24 oct 2025	20 oct 2025, 6:23		⚖ Medium

Figura 21: Sprint 3 de tareas: Optimización y pruebas.

4. **Últimos detalles y evaluación del rendimiento:** Una vez realizado todo el código y solucionados los problemas y bugs hasta el momento, esta última sección de trabajo se centró principalmente en realizar la correspondiente evaluación de rendimiento y energía, asegurar el correcto funcionamiento de los tests unitarios y funcionales y refactorizar completamente el código para asegurar el correcto cumplimiento de las restricciones de *clang tidy*.

Actividad	Persona asignada	Informador	Fecha de vencimi...	Creada	↓	Prioridad
✦ KAN-25 ULTIMOS DETALLES Y EVALUACIÓN D...	Sin asignar	Noa López Fern...	30 oct 2025	27 oct 2025, 13:19		⚖ Medium
☒ KAN-30 Conclusiones y mejora de la memoria	Sin asignar	Noa López Fern...	Ninguno	27 oct 2025, 13:21		⚖ Medium
☒ KAN-29 Evaluaciones de rendimiento y energía	Sin asignar	Noa López Fern...	Ninguno	27 oct 2025, 13:20		⚖ Medium
☒ KAN-28 Realización de pruebas funcionales	Sin asignar	Noa López Fern...	Ninguno	27 oct 2025, 13:20		⚖ Medium
☒ KAN-27 Revisión de pruebas unitarias	Sin asignar	Noa López Fern...	Ninguno	27 oct 2025, 13:20		⚖ Medium
☒ KAN-26 Refactorizaciones necesarias para cla...	Sin asignar	Noa López Fern...	Ninguno	27 oct 2025, 13:19		⚖ Medium

Figura 22: Sprint 4 de tareas: Últimos detalles y evaluación del rendimiento.

5.2. Desarrollo de tareas

Además de los sprints presentados en la sección anterior que presentan las principales tareas a realizar de manera general, la subdivisión en subtareas dentro de dichos sprints, hemos generado una hoja de cálculo en la que se especifican detalladamente la realización de las diferentes subtareas por los diferentes miembros del grupo agrupadas por sprint, así como las horas dedicadas a cada una y la fecha de realización. Dicha hoja de cálculo se puede encontrar en el siguiente link: [Organización de trabajo del equipo 10](#)

6. Conclusiones

Como hemos podido comprobar en la sección de *Evaluación de rendimiento y energía*, las optimizaciones realizadas han conseguido mejorar notablemente el rendimiento de nuestro proyecto. De hecho, como ya se ha mencionado, nuestros resultados de tiempo de ejecución son notablemente inferiores a los presentados como referencia en la plataforma *Aula Global*. Y ya no solo eso, también presentamos unos resultados de consumo energético y rendimiento que, para el crecimiento de objetos a renderizar crece de forma controlada y lineal, lo cual es algo que demuestra la positividad de los efectos de nuestras optimizaciones, como ya se ha mencionado en secciones anteriores de la memoria.

Por otro lado, en cuanto a la arquitectura de nuestro software, hemos podido realizar una división correcta entre las secciones de código que corresponden al modelo *AOS* al *SOA* y a aquellos ficheros comunes entre ambos modelos. Esto ha sido un aspecto clave en la realización de nuestro proyecto, ya no solo porque nos ha permitido dividir el trabajo de una manera más eficiente y organizada que nos ha permitido trabajar en paralelo para poder llegar a las fechas requeridas, sino también porque nos ha permitido aplicar las optimizaciones de una manera verdaderamente sencilla. Es más, nuestra división de las secciones comunes y las específicas (explicadas en mayor profundidad en la sección de *Diseño*) nos ha permitido evitar el código duplicado, que ha resultado también en permitir que cualquier optimización o cambio se aplicase tanto al modelo *AOS* como al *SOA* de manera efectiva sin necesidad de duplicar código.

En relación con esto, consideramos realmente que este trabajo no solo nos ha planteado retos en relación con la *Arquitectura de Computadores*, ayudandonos a comprender mejor los conceptos vistos en clase, los métodos de optimización y los aspectos relacionados con el análisis de rendimiento y consumo energético y la realización de conclusiones pertinentes. El trabajo, además nos ha presentado retos a nivel *Arquitectura del Software* lo que nos ha permitido mejorar nuestras capacidades de organización de diseño, de trabajo conjunto para el desarrollo de software y el entendimiento de un lenguaje de programación con el que no habíamos tenido un extenso contacto previo, como es *C++*.

Por otro lado, el trabajo nos ha permitido conectar con mecanismos de optimización que se usan en industrias como es la del desarrollo de videojuegos. Precisamente en nuestro caso, una de las principales optimizaciones presentadas en nuestro código es la implementación de *Axis Aligned Bounding Boxes (AABB)*, una técnica que se utiliza de manera extensa en la industria de los videojuegos para permitir la mejora de rendimiento de las intersecciones entre rayos y entre elementos dentro de las escenas. Hemos entendido perfectamente el uso de dicho modelo pues hemos comprobado la eficacia que este supone para la mejora del rendimiento en nuestro proyecto concreto. Ya no solo eso, además nos ha permitido presentar unos resultados completamente escalables, pues sus efectos en la medida de rendimiento y consumo, como se ha visto en el comienzo de la sección de *Evaluación del Rendimiento* nos permite comprobar que la optimización es aún más efectiva cuando el número de elementos que interviene en la escena es mayor. Consideramos, que la escalabilidad es una característica fundamental que se debe tener en mente en cualquier diseño software, y en este caso, hemos conseguido encontrar mecanismos que nos permiten garantizarla.

Por otro lado, pese a no haber podido sacar unas conclusiones definitivas en relación con la diferencia de rendimiento en nuestro proyecto dependiendo del uso de *AOS* o *SOA*, pues el efecto de nuestras optimizaciones ha conseguido que la diferencia sea prácticamente despreciable, consideramos que la práctica nos ha sido completamente útil para comprender mejor el funcionamiento de ambos modelos y sus aplicaciones en cuanto a código se refiere. Además, hemos podido trabajar en base a la transformación de un modelo al otro, pues, de manera general, aplicábamos nuestros cambios únicamente en uno de los dos modelos indistintamente, y una vez probábamos la eficacia y correcto funcionamiento de los cambios, lo traducíamos al modelo contrario. Esto también nos ha permitido comprender mejor las diferencias entre ambos y mejorar nuestra capacidad de modificación y adaptación del código.

Además, durante la realización del proyecto nos hemos enfrentado a numerosos problemas y retos. Entre estos retos, el primero de todos fue la optimización del código relacionado con los cilindros. Como se ha mencionado en secciones anteriores, el código relacionado con las intersecciones y generaciones de rayos en cilindros suponía más de un 67% del código en nuestra versión inicial. Para nosotros, eso fue un reto verdaderamente importante, pues la primera solución encontrada, el precalculo de los valores relacionados, nos proporcionó una solución efectiva, pero nos abrió las puertas a la gestión de memoria caché, una cuestión completamente nueva para nosotros. No obstante, gracias a ello, conseguimos también relacionar los conceptos vistos en clase acerca de la misma a nuestra práctica y profundizar más en la relación entre la práctica y el contenido teórico que estudiamos en clase. Por otro lado, otro de los mayores retos a los que nos enfrentamos fue la resolución de *bugs* en el código, especialmente dos: el correcto funcionamiento de los cilindros, pues estos nos presentaban unas extrañas manchas negras a las que no conseguíamos encontrar explicación, y también el correcto funcionamiento de la lógica relacionada con los materiales refractivos. Estos dos *bugs* nos presentaron un verdadero reto, y realmente ocuparon una parte importante del tiempo dedicado al proyecto, principalmente durante las últimas dos semanas de trabajo. No obstante, el enfrentamiento a dichos problemas nos ha permitido adquirir conocimientos, que aunque no parecen directamente relacionados con la asignatura, nos permiten adquirir competencias que realmente están relacionadas con el mundo laboral. Entre ellas, la revisión de código de compañeros, pues en la mayoría de las ocasiones, la búsqueda del problema o solución en el código la realizábamos sobre el código realizado por nuestros compañeros, para mediante un enfoque nuevo, conseguir soluciones efectivas. En ese mismo sentido, la práctica nos ha permitido mejorar nuestras capacidades de uso de herramientas de IA Generativa, principalmente para la depuración de código, lo cual consideramos es una competencia que será de gran utilidad en el futuro laboral próximo dada la creciente importancia de estas herramientas.

En definitiva, consideramos que la realización de este proyecto ha sido muy positiva para nuestro aprendizaje, no solo en relación a los contenidos de la asignatura, también en relación a aspectos relacionados con nuestro futuro laboral y con las funciones cotidianas de los ingenieros informáticos. Para nosotros, eso es algo que se valora muy positivamente y consideramos por ello que la práctica nos ha sido de verdadera utilidad. Además consideramos que el trabajo ha sido completamente satisfactorio, obteniendo unos resultados de rendimiento y consumo energético bastante buenos y adquiriendo conocimientos verdaderamente variados y útiles para nuestro futuro.

7. Uso de herramientas de IA

En el desarrollo de este proyecto, y conforme a las directrices establecidas en el enunciado y en los estándares establecidos en el departamento, a lo largo de la realización del proyecto, hemos empleado diversas herramientas de Inteligencia Artificial como apoyo para la productividad, la resolución de problemas y la consolidación de los conocimientos adquiridos.

A continuación, se detalla el rol específico que cada modelo ha desempeñado en nuestro flujo de trabajo, destacando, como es evidente, que la responsabilidad final sobre el diseño, la corrección y el rendimiento del código ha recaído siempre sobre los integrantes del equipo.

Pese a que el uso de dichas herramientas ha variado en base al integrante del grupo, en términos generales, se han empleado las siguientes herramientas.

1. Gemini 2.5 Pro: Asistente de Desarrollo Integral:

Gemini 2.5 Pro fue la herramienta más utilizada a lo largo del proyecto con un uso principalmente concentrado en tres áreas clave para el desarrollo del proyecto:

- Consultoría de los primeros conceptos necesarios para la realización del proyecto, proporcionando ayuda en temas con los que no nos sentíamos muy familiarizados.
- Ayuda de diseño y *brainstorming* de desarrollo del proyecto.
- Principal herramienta de desarrollo y comprobaciones de código rápido, dada su capacidad para grandes ventanas de contexto.

No obstante, en ocasiones su uso resultaba desesperante al no ser el mejor LLM para el depurado de código, y en casos de asistencia para solucionar *bugs* podía resultar notoriamente inefectivo. Es más, tras consultar acerca de uno de los mayores errores que enfrentamos en la realización del código, la herramienta repetía en bucle las mismas ideas erróneas para solucionarlo, que habíamos comprobado previamente como no funcionales. No obstante, fuimos capaces de identificar el error tras muchos intentos, guiando a la herramienta hacia los casos que nosotros veíamos como más factibles.

2. Deepseek: Especialista en Problemas Complejos:

Recurrimos a Deepseek de manera notoriamente más selectiva, principalmente cuando nos enfrentábamos a complejos *bugs* que resultaban incapaces de solucionar con otros LLM. Su uso viene condicionado principalmente por la velocidad de respuesta de dicha herramienta. Su lentitud de razonamiento lo hace muy eficaz para la resolución de problemas más complejos y el depurado de código, no obstante, la pérdida de agilidad que supone limita su uso a casos aislados.

Un símil que solemos usar en el equipo para explicar dichas diferencias con respecto a los otros LLMs utilizados durante el proyecto es el de un francotirador: certero y preciso (Deepseek), pero sin la cadencia, comodidad ni agilidad de una metralleta (Gemini).

3. ChatGPT 5: Tutor Conceptual y Clarificador Ágil de Código:

El rol de ChatGPT 5 fue mayoritariamente pedagógico y lo empleamos como un tutor para afianzar nuestra comprensión de conceptos teóricos concretos y como un asistente que permitía analizar pequeños fragmentos de código ya escritos.

En concreto, este LLM es muy veloz, no obstante, con el plan de uso gratuito, el uso está muy limitado tanto en número de prompts, longitud de mensajes y capacidad para proporcionar contexto. Por ello, aunque ha sido usado en momentos puntuales, su uso para el desarrollo o revisión de código ha sido prácticamente nulo y su uso se ha reducido más a un contexto explicativo.

8. Bibliografía

- [1] CppCon, *Learn C++ Performance Secrets from a Bloomberg Engineer At CppCon 2025!*, Youtube, 24 agosto 2025 [Online] URL: <https://youtu.be/hyZ3gEGVvC8?si=Njp9NAfaxhsJGhy9>
- [2] Y. Bonenberger (CppCon), *Designing Fast and Efficient List-like Data Structures* Youtube, 13 marzo 2024 [Online] URL: https://youtu.be/stfEry0zz5E?si=Pf4h9k_m9RTxTdpB
- [3] N. Barker, *Introducción al diseño orientado a datos para juegos*, Youtube, 13 septiembre 2023 [Online] URL: https://youtu.be/WwkuAqObplU?si=bEvw3x2wVWt_SAtc
- [4] G. Gambetta, *Computer Graphics from Scratch: A Programmer's Introduction to 3D Rendering* 1ed. , USA: No Starch Press 13 mayo 2021 [Online] URL: <https://gabrielgambetta.com/computer-graphics-from-scratch/>
- [5] Y. Sharvit, *Principles of Data-Oriented Programming* , 22 junio 2022 [Online] URL: <https://blog.klipse.tech/dop/2022/06/22/principles-of-dop.html>
- [6] S. Kennedy, *C++ Data-Oriented Design Paradigm*, CSE 491 Michigan State University, 5 marzo 2020 [Online] URL: <https://mmore500.com/cse-491/blog/2020/03/05/data-oriented-design.html>
- [7] M. Rubloff *What is an AABB?* , Radiance Fields, 10 abril 2023 [Online] URL: <https://radiancefields.com/what-is-a-aabb-for-nerfs>
- [8] MND, *3D collision detection* [Online] URL: https://developer.mozilla.org/en-US/docs/Games/Techniques/3D_collision_detection
- [9] Toriomg, *OpenGL* , Github, 2025. [Online] URL: <https://github.com/Torionmg/OpenGL>
- [10] Guidelines de formato IEEE para escritura de artículos.[Online] URL: https://revistas.pucp.edu.pe/imagenes/electro/ee_formato_ieee.pdf